Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Server-Side Request Forgery (SSRF)

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

What is a Server-Side Request?

A server may need to perform some internal/external connections to serve the client request. For instance:

- **services to expose commands**: DNS request, ICMP request
- **authentication** via, e.g., Single-Sign On (SSO): a request to the identify provider
- **captcha verification**: a request to validate the user response
- **REST API**:
  - the backend may use some external services
  - the backend may use some internal services

Server-side request forgery (SSRF) in a nutshell

What is Server-Side Request Forgery (SSRF)?

When the request or some of its aspects can be manipulated by the user then an attacker may be able to forge an “unexpected” request:

- the end target (URI) of the request is under the control of the user:
  - FROM https://auth.service/ TO https://**attacker.com** [control the response]
  - FROM https://auth.service/ TO https://**local.ip**/ [map/access the internal network]
  - FROM https://auth.service/secret-token TO https://**attacker.com**/secret-token [data leak]
- the data sent are under the control of the user:
  - FROM https://social.com/newpost=AAA TO https ://social.com/newpost=**cat /etc/passwd**

Other consequences of SSRF

Some internal services may be sometimes accessible without any authentication when the request is coming from the internal network. Via SSRF, we may thus able to freely access the service. Examples:

- admin panels
- databases
- log handlers
- infrastructure monitors

Other consequences of SSRF

Some internal services may be sometimes accessible without any authentication when the request is coming from the internal network. Via SSRF, we may thus able to freely access the service. Examples:

- admin panels
- databases
- log handlers
- infrastructure monitors

Cloud providers may expose sensitive metadata through specific internal hosts.

- E.g., AWS exposes instance metadata at http://169.254.169.254 which may leak sensitive information.

Blind SSRF

In some cases, the server will not provide any explicit feedback about the request: e.g., the response of the server-side request is not shown to the user and we are unable to perform external connections.

We can still perform nasty things with a blind SSRF:

- get a feedback by looking at the time required for the server-side request:
  - e.g., the server-side request may take a different time depending on the internal host (invalid or not) and port (open or closed) that we are testing.
- blindly execute requests/actions in the internal lan

Preventing SSRF

- **whitelist approach**: requests are made only towards specific hosts. Hard to do when we want to allow connections even to unexpected hosts.

Preventing SSRF

- **whitelist approach**: requests are made only towards specific hosts. Hard to do when we want to allow connections even to unexpected hosts.
- **isolated host**: the host performing the request should be isolated from the rest of the network and should not have access to any sensitive data

302Camo

Web Challenge XX

302Camo - Reconnaissance

What can we observe? The challenge...

- presents a page to sign-up and sign-in into a blog system;
- allow the user to download the source code of the system.

302Camo - Reconnaissance

What can we observe? The challenge...

- presents a page to sign-up and sign-in into a blog system;
- allow the user to download the source code of the system.

> Maybe the source code can tell us something?

302Camo - Hypothesis

In the source code there is a file called `get_flag.php` and here is the content:

```php
<?php

if ($_SERVER['REMOTE_ADDR'] == '127.0.0.1') {
    echo getenv('FLAG');
} else {
    echo 'lol nope';
}
```

302Camo - Hypothesis

In the source code there is a file called `get_flag.php` and here is the content:

```php
<?php

if ($_SERVER['REMOTE_ADDR'] == '127.0.0.1') {
    echo getenv('FLAG');
} else {
    echo 'lol nope';
}
```

> Who is authorized to read the flag?

302Camo - Hypothesis

In the source code there is a file called `get_flag.php` and here is the content:

```php
<?php

if ($_SERVER['REMOTE_ADDR'] == '127.0.0.1') {
    echo getenv('FLAG');
} else {
    echo 'lol nope';
}
```

Who is authorized to read the flag?

Are there any other pages that could be useful?

PAAS

Web Challenge 05

PAAS - Reconnaissance

What can we observe? The challenge...

- presents a terminal-like interface in the browser with a single available command: `pi`;
- the `pi` command accepts a number as argument and returns digits of pi;

PAAS - Hypothesis

Inspecting the page source reveals that the input is encoded with `btoa()` (base64) and sent via POST to a `/data` endpoint - could the server be dynamically evaluating our input...?